Project: Dự đoán bệnh tiểu đường

Môn học: Phát triển các hệ thống thông minh

Năm 2026

Mục lục

Phạm vi và lưu ý **iv**

1 Giới thiệu **1**

1.1 Bài toán và động lực . . . 1
1.2 Mục tiêu và phạm vi theo gợi ý của giảng viên . . . 1
1.3 Phát biểu bài toán . . . 1
1.4 Phạm vi, yêu cầu và tiêu chí đánh giá . . . 2
1.5 Đối tượng sử dụng và kịch bản . . . 2
1.6 Đóng góp và giới hạn của đồ án . . . 2

2 Hệ đa agent **3**

2.1 Agent . . . 3
2.2 Hệ đa agent và vai trò trong Đường An . . . 3
2.3 Mô hình hóa agent, trạng thái và điều phối . . . 3
2.4 Đồ thị LangGraph cho yêu cầu hỏi đáp . . . 4
2.5 Hợp đồng trao đổi và quy tắc xử lý lỗi . . . 4
2.6 Ví dụ đường đi của một yêu cầu . . . 4

3 Kiến trúc tổng quan **5**

3.1 Vai trò của các agent và sơ đồ triển khai . . . 5
3.2 Luồng một lượt hỏi đáp và các node chạy . . . 5
3.3 Phân tầng triển khai và luồng dữ liệu . . . 5
3.4 Bảng endpoint và hợp đồng . . . 6
3.5 Lưu trữ và cấu trúc đồ thị . . . 6
3.6 Mô hình triển khai và các điểm tin cậy . . . 7

4 Các thành phần của hệ thống **7**

4.1 Deep Learning và Prediction Agent . . . 7
4.2 Thực nghiệm notebook và lựa chọn mô hình . . . 11
4.3 Knowledge Base và RAG . . . 12
4.4 Reinforcement Learning: tác tử gợi ý học tập . . . 13
4.5 Agent luật an toàn . . . 14

4.6 Điều phối và kết quả tự động 15

5 Code và Demo **15**

5.1 Cấu trúc mã nguồn 15

5.2 Các hàm chính và quan hệ với giao diện 15

5.3 Cách chạy và tái lập 16

5.4 Demo giao diện và triển khai 17

5.5 Notebook thực hành 22

5.6 Giải thích kết quả, phản biện và độ tin cậy 23

5.7 Kiểm thử và đánh giá kết quả 23

5.8 Đánh giá và hạn chế 24

5.9 Hướng phát triển 24

6 Kết luận **25**

Tài liệu tham khảo **25**

Danh sách hình vẽ

1 Kiến trúc hiện thực: luồng hỏi đáp, phân loại và mô phỏng là các nhánh API riêng . 5
2 Ma trận nhầm lẫn MLP; tính từ số liệu lưu sau huấn luyện, không phải ảnh một lần chạy mới 8
3 Tổng thưởng trung bình qua năm nhóm seed, lấy từ báo cáo RL đã lưu 14
4 Trang chủ Đường An chụp từ ứng dụng đang chạy 17
5 Giao diện nhập câu hỏi và các câu hỏi gợi ý 18
6 Ví dụ hỏi đáp thực tế: nguồn web chưa duyệt được hiển thị dạng liên kết; trace cho thấy các bước đã chạy 19
7 Giao diện phiên bản trước khi bổ sung biểu mẫu NHANES 15 chỉ số 20
8 Giao diện mô phỏng lựa chọn chủ đề giáo dục 20
9 Kết quả phân loại BRFSS với dữ liệu giả lập và trace Prediction Agent 21
10 Kết quả chọn chủ đề DQN với trạng thái giả lập và trace Learning Agent 22

Phạm vi và lưu ý

Đường An là nguyên mẫu học thuật minh họa kiến trúc đa tác tử, phân loại dữ liệu, truy xuất tri thức và học tăng cường. Kết quả không phải chẩn đoán, không dùng để kê đơn hoặc thay thế bác sĩ. Nội dung tóm tắt thủ công chưa được thẩm định chuyên môn độc lập. Không nhập thông tin sức khỏe hoặc định danh thật khi demo.

Báo cáo dùng kết quả huấn luyện đã lưu trong `artifacts/*/report.json`; không khẳng định đã huấn luyện lại khi viết báo cáo. Các ảnh giao diện được chụp từ ứng dụng đang chạy ngày 04/11/2026, dùng câu hỏi hoặc dữ liệu giả lập. Ảnh chụp là minh chứng lịch sử; cấu hình và kết quả notebook mới được ghi rõ ở phần thực nghiệm.

1 Giới thiệu

1.1 Bài toán và động lực

Bệnh đái tháo đường liên quan đến nhiều nội dung cần tìm hiểu như biểu hiện, yếu tố nguy cơ, dinh dưỡng và chăm sóc. Mục tiêu đồ án là xây dựng một nguyên mẫu giúp trình bày thông tin có nguồn, khảo sát khả năng phân loại trên dữ liệu nghiên cứu và minh họa lựa chọn nội dung giáo dục thông qua học tăng cường. Dự án kết hợp giao diện web, các quy tắc an toàn, cơ sở tri thức đồ thị và các mô hình học máy.

Các chức năng thực hiện: hỏi đáp dựa trên tài liệu, hiển thị đường dẫn nguồn và vết xử lý; thử nghiệm phân loại UCI, BRFSS và NHANES bằng MLP/CNN; mô phỏng gợi ý chủ đề bằng DQN; kiểm tra tình trạng dịch vụ và triển khai bằng Docker Compose.

1.2 Mục tiêu và phạm vi theo gợi ý của giảng viên

Báo cáo trình bày bài toán, hệ đa agent, kiến trúc tổng quan, các thành phần Deep Learning, cơ sở tri thức và RAG, học tăng cường, luật an toàn, cùng mã nguồn và minh chứng demo. Sơ đồ ở phần 3 thể hiện các thành phần đang được triển khai trong hệ thống.

1.3 Phát biểu bài toán

Trong đồ án này, đầu vào không được giả định là một bệnh án có cấu trúc duy nhất. Người dùng có thể đặt câu hỏi ngôn ngữ tự nhiên, điền một biểu mẫu đặc trưng của bộ dữ liệu hoặc lựa chọn nhu cầu học tập. Ba loại đầu vào này có ý nghĩa khác nhau và không nên gộp thành một điểm số chung:

- **Hỏi đáp tri thức:** đầu vào là câu hỏi tiếng Việt và một phần ngữ cảnh hội thoại; đầu ra là câu trả lời có danh sách nguồn, hoặc phản hồi an toàn/thiếu tài liệu. Chất lượng phụ thuộc vào độ phù hợp của tài liệu truy xuất và độ trung thực của quá trình sinh.
- **Phân loại dữ liệu bảng:** đầu vào phải khớp với hợp đồng đặc trưng của một mô hình UCI hoặc BRFSS; đầu ra là điểm sigmoid, lớp theo ngưỡng và chú thích về giới hạn. Nhãn hai bộ không có cùng cách thu thập.
- **Gợi ý học tập:** đầu vào là bốn giá trị nhị phân biểu thị các chủ đề chưa học; đầu ra là tên chủ đề được chọn trong một môi trường giả lập. Đây không phải đề xuất thuốc hoặc quyết định điều trị.

Như vậy nhiệm vụ hệ thống có thể xem là ánh xạ $f(x, t) \rightarrow y$, trong đó t xác định loại tác vụ. Không tồn tại bằng chứng trong mã hoặc dữ liệu cho phép coi y là kết luận chẩn đoán lâm sàng. Việc thiết kế API riêng cho từng tác vụ giúp xác thực hợp đồng đầu vào và giải thích ý nghĩa đầu ra rõ ràng hơn.

1.4 Phạm vi, yêu cầu và tiêu chí đánh giá

Nhóm	Yêu cầu	Bằng chứng cần đối chiếu
Chức năng	Hỏi đáp, phân loại, mô phỏng giáo dục	Ảnh giao diện; kết quả API; trace từng nhánh.
Dữ liệu	Xác định nguồn, biến, nhãn và phép chia	`docs/DATA.md`; mã tiền xử lý; report JSON.
Hiệu năng	Có phương pháp đánh giá và baseline	Ma trận nhầm lẫn, precision, recall, ROC-AUC; heuristic cho DQN.
An toàn	Có nhánh xử lý một số đầu vào đặc biệt	Rule Agent và kiểm thử ca cụ thể; không suy rộng cho mọi trường hợp.
Vận hành	Có hướng dẫn chạy và thông tin deploy	Compose, health endpoint, URL, kết quả kiểm tra tại thời điểm báo cáo.

Các tiêu chí trên giúp kiểm tra nguyên mẫu học thuật. Chúng không tương đương tiêu chuẩn công nhận một sản phẩm y tế; triển khai thực tế còn cần kiểm định độc lập, quản trị dữ liệu và quy trình chuyên môn phù hợp.

1.5 Đối tượng sử dụng và kịch bản

Người trình bày có thể lần lượt giới thiệu câu hỏi về kiến thức, một biểu mẫu giả lập và bài toán chọn chủ đề giáo dục. Người xem có thể đối chiếu nguồn được hiển thị, vị trí node trong mã và kết quả JSON. Toàn bộ ví dụ trong báo cáo đều là dữ liệu giả lập hoặc số liệu đã tổng hợp từ tập nghiên cứu, không phải hồ sơ người bệnh cụ thể.

Với câu hỏi ngắn liên quan đến tiểu đường, đường đi dự kiến gồm kiểm tra luật, truy xuất và phản hồi. Với câu hỏi yêu cầu điều chỉnh liều, nhánh chặn trả thông báo cố định. Hai kịch bản này minh họa cơ chế rẽ nhánh, song không chứng minh độ bao phủ của bộ luật đối với mọi cách diễn đạt tiếng Việt.

1.6 Đóng góp và giới hạn của đồ án

Điểm tích hợp của đồ án nằm ở việc kết nối ba dòng công việc trong cùng giao diện và cung cấp các tệp tái lập: mã backend, mã huấn luyện, dữ liệu định nghĩa nguồn, artifact và notebook trình bày. Giá trị của đồ án là cho phép quan sát mối quan hệ giữa dữ liệu, mô hình, tri thức và điều phối tác vụ. Không có đánh giá người dùng thật, thử nghiệm lâm sàng hoặc chứng cứ khái quát hóa sang dân số Việt Nam trong phạm vi hiện tại.

2 Hệ đa agent

2.1 Agent

Agent là thành phần nhận đầu vào, xử lý theo chức năng được giao và trả lại kết quả. Hệ đa agent phối hợp nhiều vai trò để giải quyết một công việc. Trong nguyên mẫu này, tên agent mô tả *vai trò xử lý*; không nên hiểu rằng mọi vai trò đều là một tiến trình độc lập hoặc một LLM tự chủ.

2.2 Hệ đa agent và vai trò trong Đường An

Vai trò	Chức năng và biểu hiện trong mã
Coordinator	Luồng hỏi đáp được mô tả bằng LangGraph: rules, blocked, knowledge, response; định tuyến theo kết quả kiểm tra. API streaming tái sử dụng các hàm node và tuần tự xuất dữ liệu.
Rule Agent	Áp dụng các mẫu/quy tắc để nhận diện một số tình huống khẩn cấp, hỏi liều, dữ liệu nhạy cảm và prompt injection. Đây không phải bộ phát hiện ngữ nghĩa đầy đủ.
Knowledge Agent	Viết lại câu hỏi tiếp nối khi có lịch sử, tạo embedding và tìm đoạn tương đồng trong Neo4j.
Response Agent	Trả lời dựa trên đoạn nhập thủ công; khi chỉ tìm thấy nguồn web chưa kiểm duyệt thì đưa liên kết thay vì tự tổng hợp.
Prediction Agent	API kiểm tra biểu mẫu, nạp bộ tiền xử lý, MLP hoặc CNN rồi trả điểm và vết xử lý.
Learning Agent	API dùng chính sách DQN chọn chủ đề trong môi trường giáo dục giả lập; có quy tắc dự phòng nếu chọn chủ đề đã hoàn tất.

2.3 Mô hình hóa agent, trạng thái và điều phối

Một agent trong hệ thống tính toán thường được mô tả qua quan sát, trạng thái, tập hành động và mục tiêu. Agent không nhất thiết phải được cài đặt bằng LLM: một hàm quy tắc hoặc một mô hình dự đoán cũng có thể đảm nhiệm vai trò tác tử trong mô hình thiết kế. Ở đồ án, các vai trò được biểu diễn chủ yếu bằng các hàm trong cùng tiến trình FastAPI, chia sẻ các hợp đồng dữ liệu; điều này khác với hệ phân tán gồm nhiều tiến trình giao tiếp độc lập.

Trạng thái hỏi đáp `CoordinatorState` gồm câu hỏi, lịch sử, kết quả kiểm tra an toàn, nhóm vi phạm, danh sách nguồn, câu trả lời và trace. Từng node nhận trạng thái hiện có và trả về các trường cần cập nhật. Thiết kế này giúp tách nhiệm vụ, nhưng bản thân trace là chuỗi do chương trình ghi nhận chứ không phải nhật ký kiểm toán độc lập chống sửa đổi.

2.4 Đồ thị LangGraph cho yêu cầu hỏi đáp

Trong endpoint đồng bộ `/api/chat`, đồ thị được biên dịch với cạnh `START` $\rightarrow$ `rules`. Nếu kết quả `safety_passed` là sai, luồng đi đến `blocked` rồi kết thúc. Một số câu chào, cảm ơn và câu hỏi Claude cụ thể được xử lý bởi nhánh `conversation`; các câu khác đi đến `knowledge` và `response`. Đây là luồng điều phối xác định, không có vòng lặp thương lượng giữa các agent.

Endpoint `/api/chat/stream` không gọi trực tiếp `coordinator.invoke`. Nó tái sử dụng các hàm node quy tắc, chặn, hội thoại, truy xuất và hàm sinh từng đoạn để xuất dữ liệu. Vì hai endpoint có cách điều phối riêng, khi thay đổi logic node cần kiểm thử cả hai để tránh hành vi lệch nhau.

2.5 Hợp đồng trao đổi và quy tắc xử lý lỗi

Câu hỏi được ràng buộc từ 2 đến 1.500 ký tự; lịch sử tối đa 10 tin nhắn, từng tin có vai trò user hoặc assistant và tối đa 1.500 ký tự. API kiểm tra danh sách đặc trưng của mô hình phải đúng bộ cột yêu cầu. Nếu thiếu mô hình, endpoint phân loại trả trạng thái 503 thay vì tự tạo dự đoán mẫu. Nếu Neo4j hoặc Ollama lỗi, luồng chat trả lỗi và giao diện báo cho người dùng.

Điểm quan trọng khi trình bày là **phản hồi từng đoạn không bảo đảm kiểm chứng từng đoạn**. Các token đã xuất ra không thể tự động thu hồi nếu mô hình sinh sai ở phần sau. Cơ chế stream chủ yếu giảm thời gian chờ cảm nhận; an toàn nội dung vẫn phụ thuộc quy tắc, tài liệu, mô hình và các bước kiểm định bổ sung.

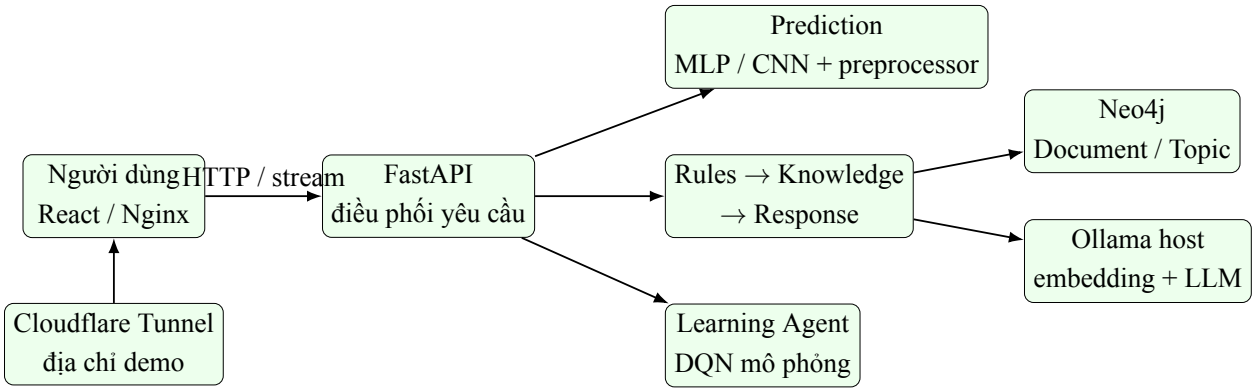
2.6 Ví dụ đường đi của một yêu cầu

Với câu hỏi “Tiểu đường là gì?” trong ảnh minh chứng, quy tắc trả trạng thái cho phép, truy xuất tìm thấy một nguồn web tự động và nhánh phản hồi chỉ hiển thị liên kết chưa kiểm duyệt. Trace hiển thị thứ tự Coordinator, Rule Agent, Knowledge Agent và Response Agent. Tài liệu được tìm thấy trong trường hợp này không đủ để kết luận hệ thống luôn trả đúng nguồn cho những câu hỏi khác; trường hợp ngoài lĩnh vực trong bộ test là một ví dụ cần kiểm tra thêm.

Với yêu cầu điều chỉnh liều, phương thức `evaluate` phát hiện mẫu liên quan, đặt trạng thái an toàn là sai và loại vi phạm là yêu cầu điều chỉnh thuốc. Node chặn tạo thông báo cố định, danh sách nguồn rỗng và không đi vào nhánh truy xuất. Trật tự các bộ lọc thực tế là prompt injection, khẩn cấp, thuốc, sau đó thông tin nhạy cảm; vì thế một câu khớp nhiều nhóm được gán nhóm xuất hiện trước theo mã.

3 Kiến trúc tổng quan

3.1 Vai trò của các agent và sơ đồ triển khai



Hình 1: Kiến trúc hiện thực: luồng hỏi đáp, phân loại và mô phỏng là các nhánh API riêng

Các container gồm web, API, MySQL và Neo4j; tunnel là dịch vụ bổ sung. Ollama chạy trên máy host. MySQL chỉ ghi thời điểm và loại sự kiện nếu bật cấu hình lịch sử; không là nguồn huấn luyện hay kho tri thức. Website không tự thu thập dữ liệu trong lúc chat: nguồn được nạp riêng bằng lệnh refresh.

3.2 Luồng một lượt hỏi đáp và các node chạy

1. Gửi câu hỏi và tối đa 10 tin nhắn lịch sử; Rule Agent kiểm tra trước khi truy xuất.
2. Nếu vi phạm quy tắc, nhánh `blocked` tạo cảnh báo, không gọi truy xuất. Nếu đạt, một số câu hội thoại đơn giản được xử lý trực tiếp; các câu khác qua `knowledge` tìm tối đa ba nguồn theo độ tương đồng.
3. `response` sử dụng nguồn nhập thủ công để tạo phản hồi có trích dẫn; nguồn tự động chưa duyệt chỉ được hiển thị như liên kết tham khảo.
4. API `/api/chat/stream` gửi từng đoạn dạng NDJSON trên HTTP; Nginx tắt buffering. Cuối luồng gửi nguồn và trace. Không phải WebSocket.

3.3 Phân tầng triển khai và luồng dữ liệu

Hệ thống gồm ba tầng chính. Tầng trình bày là React/TypeScript được build thành tệp tĩnh và phục vụ bởi Nginx. Nginx chuyển các đường dẫn `/api/` đến FastAPI, đồng thời tắt buffering để trình duyệt nhận dần dữ liệu stream. Tầng ứng dụng cung cấp API xác thực đầu vào, điều phối và nạp

model. Tầng dữ liệu và mô hình gồm Neo4j, các artifact trên volume, Ollama ở host và MySQL tùy chọn cho metadata sự kiện.

Compose gắn các thư mục `artifacts`, `data`, `knowledge`, `ml`, `backend` vào container API. Mô hình và dữ liệu nhạy về dung lượng không cần nằm trong image. Các cổng công bố trên host là 8080 cho web, mặc định 8001 cho API, 7474 cho giao diện Neo4j; cổng API có thể đổi bằng biến `API_HOST_PORT`. Các cổng này gắn vào loopback, không mở trực tiếp cho mạng LAN. Truy cập từ bên ngoài được cấu hình qua Cloudflare Tunnel đến web.

3.4 Bảng endpoint và hợp đồng

Endpoint	Đầu vào chính	Kết quả và vai trò
GET /api/health	Không có body	Trạng thái API, khả năng kết nối Neo4j/Ollama, sự tồn tại tệp artifact. Kiểm tra tệp tồn tại không đồng nghĩa suy luận thành công.
POST /api/predict	kind, features	Điểm model, lớp nhị phân, ngưỡng 0,5, trace và giới hạn.
POST /api/chat	question, history	Câu trả lời hoàn chỉnh, nguồn và trace; sử dụng đồ thị LangGraph.
POST /api/chat/stream	question, history	Dòng NDJSON chứa sự kiện delta, done hoặc error; giao diện ghép các đoạn khi nhận.
POST /api/learning	needs gồm bốn cờ 0/1	Chủ đề hoặc trạng thái hoàn thành; có thể ghi nhận việc override.

3.5 Lưu trữ và cấu trúc đồ thị

Khi nạp tài liệu thủ công, chương trình tạo ràng buộc duy nhất cho ID Document và tên Topic; mỗi Document có văn bản, tiêu đề, URL, chủ đề và embedding. Với nguồn web, một WebSource có thể liên kết với nhiều Document là các đoạn của trang. Việc phân đoạn đặt giới hạn chiều dài 1.200 ký tự; giới hạn này là lựa chọn kỹ thuật, không bảo đảm ranh giới y khoa hoặc ngữ nghĩa tối ưu.

Chương trình refresh tính hash nội dung trang sau phân đoạn để tránh cập nhật lại khi nội dung không đổi. Nếu nội dung thay đổi, nó tạo xong tất cả embedding trước khi xóa và tạo các đoạn mới trong transaction Neo4j; khi quá trình tải hoặc tạo embedding thất bại, dữ liệu cũ được giữ lại. Lịch cập nhật thực tế trong Compose hiện dùng profile chạy thủ công; không mặc định có bộ lập lịch chạy nền.

3.6 Mô hình triển khai và các điểm tin cậy

Các thành phần giao tiếp qua mạng Docker nội bộ; Ollama được gọi bằng địa chỉ host. Browser chỉ thấy giao diện và API qua Nginx. Cấu hình tunnel lấy từ thư mục cục bộ không đưa vào báo cáo hoặc mã nguồn công khai. Rate limiter hiện được lưu trong bộ nhớ theo IP quan sát tại ứng dụng, 60 yêu cầu mỗi 60 giây cho đường dẫn API; trong môi trường proxy hoặc nhiều worker, IP và bộ đếm có thể không phản ánh chính xác từng người dùng. Đây không phải cơ chế bảo vệ chống tấn công toàn diện.

Lịch sử hội thoại được giữ trong phiên trình duyệt. MySQL, nếu tùy chọn lưu sự kiện được bật, chỉ ghi loại sự kiện và thời điểm theo thiết kế của `history.py`; không nên suy ra rằng mọi log, proxy hoặc các thành phần bên ngoài đều đã được đánh giá quyền riêng tư.

4 Các thành phần của hệ thống

4.1 Deep Learning và Prediction Agent

4.1.1 Dữ liệu, tiền xử lý và chia tập

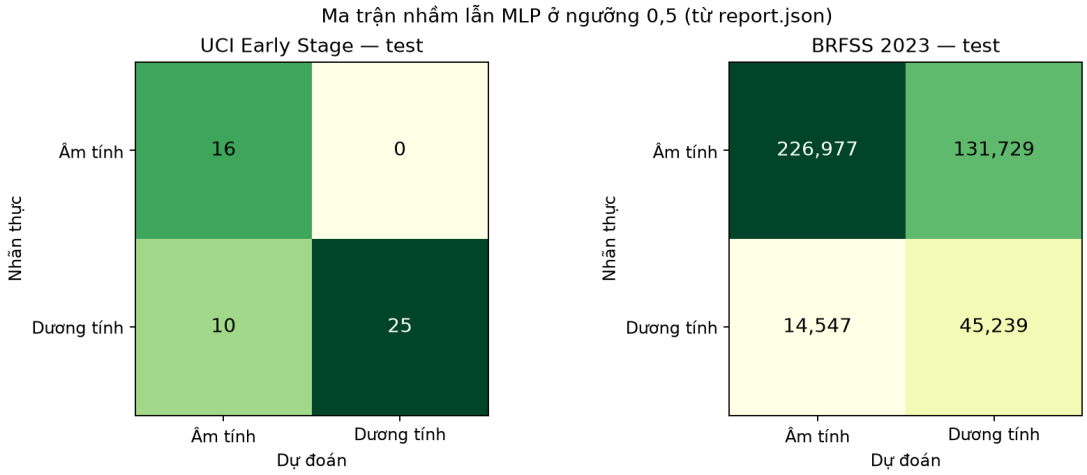
UCI Early Stage có 520 dòng ban đầu; sau loại trùng còn 251 hồ sơ khác nhau, chia train/validation/test 160/40/51. Tập thuộc một cơ sở ở Bangladesh và rất nhỏ. BRFSS là khảo sát cắt ngang tự báo cáo. Bản hiện tại giữ nhãn đã từng được báo mắc (mã 1) hoặc không mắc (mã 3), loại các mã khác; không phải nhãn dự báo mắc mới. Dữ liệu đã lọc năm 2021 và 2022 được ghép, chia train/validation; 2023 làm test theo thời gian. Không dùng survey weight và chưa chứng minh người tham gia không trùng giữa các năm.

Mỗi nhánh có bộ tiền xử lý riêng: điền thiếu, chuẩn hóa đặc trưng số, one-hot cho đặc trưng phân loại. Chỉ fit tiền xử lý trên train. MLP có hai lớp ẩn 64 và 32 nút, ReLU, dropout 0,2; Adam, BCE có trọng số lớp và early stopping theo validation. Logistic Regression cân bằng lớp làm baseline. Điểm MLP không được xem là xác suất chẩn đoán đã hiệu chỉnh.

4.1.2 Kết quả test của artifact MLP cũ

Tập test	Phương pháp	Số mẫu	Accuracy	Recall (+)	ROC-AUC
UCI	MLP	51	0,804	0,714	0,923
UCI	Logistic Regression	51	0,824	0,771	0,913
BRFSS 2023	MLP	418.492	0,650	0,757	0,764
BRFSS 2023	Logistic Regression	418.492	0,652	0,754	0,762

Bảng 4: Kết quả từ artifacts/*/report.json; ngưỡng phân loại 0,5



Hình 2: Ma trận nhầm lẫn MLP; tính từ số liệu lưu sau huấn luyện, không phải ảnh một lần chạy mới

Trong BRFSS, precision lớp dương của MLP chỉ khoảng 0,256 và có 131.729 dương tính giả. Trên UCI, chỉ 51 trường hợp test nên kết quả biến động lớn; Logistic Regression có accuracy cao hơn MLP trong lần chạy đã lưu. Không khẳng định MLP vượt baseline hoặc dùng kết quả này cho chẩn đoán.

4.1.3 Định nghĩa nhãn và ý nghĩa của hai bài toán

Bộ UCI mô tả 16 thuộc tính gồm tuổi, giới và các triệu chứng/cờ được thu thập tại một cơ sở. Tập gốc có hai nhãn Positive/Negative; sau chuẩn hóa chữ và loại bản ghi không có nhãn, chương trình loại nhóm có cùng đặc trưng nhưng nhãn xung đột, rồi loại hồ sơ trùng đặc trưng. Báo cáo artifact ghi không có dòng xung đột và còn 251 hồ sơ; đây là số hồ sơ đặc trưng khác nhau, không khẳng định là 251 cá nhân đã xác minh danh tính.

BRFSS sử dụng các năm khảo sát riêng. Mã giữ `DIABETE4=1` và `DIABETE4=3`, mã hóa lần lượt thành 1 và 0. Các giá trị khác bị loại khỏi bài toán nhị phân. Mục tiêu là phân loại trạng thái từng được báo mắc theo khảo sát; do cách thu thập cắt ngang, không nên gọi đây là mô hình dự báo thời điểm một người sẽ mắc bệnh trong tương lai. Các số lượng 423.958, 429.880 và 418.492 trong báo cáo JSON là số dòng sau lọc nhãn, không phải tổng số hồ sơ thô của từng tệp XPT.

Biến	Kiểu sau xử lý	Ý nghĩa trong pipeline
`_BMI5`	Số	Chia 100; ngoài khoảng 10–100 chuyển thành thiếu.
`_AGEG5YR`	Phân loại	Nhóm tuổi từ 1 đến 13.
`SEXVAR`	Phân loại	Giá trị 1 hoặc 2 theo bộ dữ liệu.
`_SMOKER3`	Phân loại	Bốn nhóm tình trạng hút thuốc.
`_TOTINDA`	Phân loại	Hai nhóm hoạt động thể chất theo khảo sát.

Bảng 5: Đặc trưng BRFSS được sử dụng; định nghĩa chi tiết phải đối chiếu codebook theo từng năm

4.1.4 Cách chia dữ liệu và ngăn rò rỉ

Với UCI, sau loại trùng, `train_test_split` giữ 20% cho test theo phân tầng nhãn; phần còn lại tiếp tục chia 80/20 cho train/validation. Với BRFSS, dữ liệu 2021–2022 tạo tập train/validation và năm 2023 giữ riêng cho test. Phép chia theo thời gian giảm nguy cơ sử dụng chính năm test khi điều chỉnh mô hình, nhưng không bảo đảm tách cá nhân giữa các năm hoặc độc lập với mọi nguồn dữ liệu ngoài.

Chỉ tập train được dùng để `fit` median imputation, standardization và one-hot encoder. Validation và test chỉ được `transform`. Biến số thiếu được điền bằng trung vị tập train; biến phân loại thiếu được điền bằng giá trị phổ biến nhất, các giá trị mới tại test được bộ mã hóa bỏ qua thay vì làm chương trình dừng. Việc lưu `preprocessor.joblib` cùng model bảo đảm giao diện suy luận sử dụng đúng phép biến đổi đã học.

4.1.5 Cơ sở mạng nơ-ron, hàm mất mát và baseline

Với vector đặc trưng sau tiền xử lý x, MLP tính logit $z = f_\theta(x)$ qua các phép biến đổi tuyến tính và ReLU, sau đó đưa về khoảng $[0, 1]$ bằng sigmoid $p = \sigma(z) = 1/(1 + e^{-z})$. Mô hình có 64 nút ẩn ở lớp đầu, dropout 0,2, 32 nút ở lớp tiếp theo và một logit đầu ra. Dropout được bật trong giai đoạn train và tắt khi đánh giá.

Do tỷ lệ hai lớp khác nhau, `BCEWithLogitsLoss` dùng trọng số dương bằng tỷ lệ số mẫu âm trên số mẫu dương trong tập train. Công thức minh họa là

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[w_+ y_i \log\sigma(z_i) + (1 - y_i)\log(1 - \sigma(z_i))\right].$$

Trọng số có thể thay đổi hành vi của điểm số và đánh đổi giữa hai loại sai; không tự làm cho đầu ra được hiệu chỉnh thành xác suất bệnh thực tế. Adam dùng learning rate 0,001, batch size 256, tối đa 60 epoch. Trọng số có validation loss tốt nhất được lưu lại; nếu validation không cải thiện sau 7 lượt thì dừng sớm.

Baseline là Logistic Regression với `class_weight='balanced'`, cùng đầu vào đã được biến đổi. Baseline giúp kiểm tra liệu MLP có thêm giá trị so với mô hình tuyến tính hay không; trong kết quả hiện có, sự chênh lệch nhỏ và phụ thuộc chỉ số/tập dữ liệu.

4.1.6 Thước đo và cách đọc kết quả

Với TP, TN, FP, FN là số dương đúng, âm đúng, dương giả và âm giả, ta có:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

$$F_1 = \frac{2PR}{P + R}, \qquad \text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

ROC-AUC đánh giá khả năng xếp hạng mẫu dương so với mẫu âm qua các ngưỡng, không trực tiếp cho biết số dương giả tại một ngưỡng cụ thể. Brier score là trung bình $(p_i - y_i)^2$; nó phản ánh sai số dự báo tổng hợp và không tự chứng minh hiệu chỉnh xác suất tốt. Cần xem đồng thời calibration curve và đánh giá ngoài trước khi diễn giải xác suất.

Ngưỡng	Precision (+)	Recall (+)	F1 (+)	Accuracy	FP
0,3	0,2035	0,9300	0,3339	0,4699	217.671
0,4	0,2235	0,8733	0,3559	0,5484	181.422
0,5	0,2556	0,7567	0,3822	0,6505	131.729
0,6	0,3037	0,5713	0,3966	0,7516	78.315
0,7	0,3652	0,3515	0,3582	0,8201	36.521

Bảng 6: Đánh đổi ngưỡng MLP trên test BRFSS 2023, trích từ report.json; chỉ phục vụ phân tích mô tả

Khi tăng ngưỡng từ 0,3 lên 0,7, recall giảm trong khi precision tăng ở bảng này. Các giá trị test đã được xem để minh họa; không được chọn ngưỡng tối ưu từ cùng tập test rồi coi đây là ước lượng độc lập. Việc chọn ngưỡng cho một ứng dụng thực tế cần tiêu chí và tập validation riêng, sau đó kiểm định trên tập chưa nhìn thấy.

4.1.7 Phân tích nhóm và giới hạn suy diễn

Báo cáo đã lưu cho BRFSS ghi ROC-AUC 0,7666 ở nhóm mã giới tính nam và 0,7614 ở nhóm nữ; ở nhóm 18–39, 40–59 và 60+ lần lượt khoảng 0,7313, 0,7315 và 0,6809. Các kết quả mô tả này không đủ chứng minh công bằng giữa các nhóm: chưa có khoảng tin cậy, chưa phân tích đầy đủ các lỗi theo ngưỡng, ảnh hưởng thiếu dữ liệu hoặc sai lệch nhãn. Các nhóm có dữ liệu thiếu có thể không xuất hiện trong các tổng nhóm.

Bộ dữ liệu cũ của assignment 03/04 có 324.372 dòng và nhãn `Diabetes_binary`, nhưng nguồn gốc bản xử lý và khả năng trùng với BRFSS hiện tại chưa được xác minh. Ở assignment 07, cột

`Outcome` được tạo trực tiếp từ `glyhb > 6.5`; nếu cho cùng biến này vào dự đoán nhãn thì bài toán bị rò rỉ quy tắc tạo nhãn. Do đó đồ án giữ hai pipeline hiện hành và chưa công bố việc thay thế bằng dữ liệu cũ.

4.2 Thực nghiệm notebook và lựa chọn mô hình

Sáu notebook trong `notebooks/training` được tổ chức theo từng bài toán: UCI, BRFSS, NHANES bốn chỉ số, NHANES 15 chỉ số, bộ assignment 02 và mô phỏng DQN. Các notebook phân loại phân tích phân phối, thiếu dữ liệu, histogram và tương quan trên tập train; làm sạch, chia train/validation/test trước khi fit bộ tiền xử lý; so sánh Dummy, Logistic Regression, HistGradientBoosting và mạng nơ-ron. Với mạng nơ-ron, ba seed 42, 43 và 44 được huấn luyện tối đa 200 epoch, tối thiểu 40 epoch, dừng sớm sau 25 epoch không cải thiện validation loss. Batch BRFSS là 4.096; số epoch giữa các dataset không tương đương số bước cập nhật. Mô hình được chọn theo ROC-AUC validation; ngưỡng F1 được chọn trên validation, sau đó chỉ đánh giá ứng viên được chọn trên test. Ba seed thay đổi khởi tạo nhưng không thay thế đánh giá trên nhiều lần chia độc lập.

Bài toán	**Chọn trên validation**	**AUC test**	**F1 test**	**Epoch thực tế; tốt nhất**
UCI	MLP seed 42	0,9196	0,8485	60/35; 60/35; 64/39.
BRFSS	HistGradientBoosting	0,7648	0,3975	77/52; 54/29; 84/59.
NHANES 4 biến	CNN seed 43	0,9491	0,7724	40/13; 42/17; 46/21.
NHANES 15 biến	CNN seed 42	0,9314	0,7549	70/45; 62/37; 56/31.
Assignment 02	Logistic Regression	0,9231	0,6364	41/16; 42/17; 48/23.

Các cặp epoch được ghi theo ba seed 42, 43, 44; ngưỡng trong test tương ứng lần lượt là 0,25; 0,60; 0,75; 0,80; 0,85. Kết quả không so sánh trực tiếp giữa những bộ có nhãn và quần thể khác nhau. Khoảng bootstrap phân tầng của AUC chưa tính thiết kế khảo sát; tập test đã được xem trong các lần chạy trước, do đó không còn là kiểm định cuối độc lập. Bảng chi tiết, đường loss và ma trận nhầm lẫn nằm trong notebook và `notebooks/training/KET_QUA_NANG_CAP.md`.

4.2.1 Dữ liệu xét nghiệm và nhãn

NHANES do CDC thu thập tại Hoa Kỳ, không phải bộ WHO hoặc dữ liệu Việt Nam. Nhãn `DIQ010` là câu trả lời từng được nhân viên y tế thông báo mắc bệnh; glucose và HbA1c đo cùng thời điểm có thể liên quan đến việc chẩn đoán hoặc điều trị trước đó. Các mô hình NHANES là phân loại hồi cứu, không dự đoán bệnh mới. Bộ assignment 02 gồm các chỉ số xét nghiệm và nhân trắc; nhãn trong assignment 07 được tạo theo `glyhb > 6.5`. Thí nghiệm dùng 12 biến số và 3 biến phân loại theo assignment 02, loại `glyhb` khỏi đầu vào, nhưng nhãn vẫn là nhãn tạo từ một xét nghiệm chứ không phải chẩn đoán độc lập.

Đối với mô phỏng RL, ba seed được huấn luyện mỗi seed 30.000 bước, đánh giá theo checkpoint 5.000 bước; so sánh cùng greedy và random trên validation. Greedy được chọn khi có reward validation bằng DQN, đạt reward trung bình test 2,026 trên môi trường giả lập. Đây không chứng minh tác động giáo dục hay y khoa.

Website ngoài các model ban đầu còn có biểu mẫu NHANES 15 chỉ số, dùng checkpoint CNN seed 42 và ngưỡng 0,80 từ notebook tương ứng. BMI được tính từ chiều cao/cân nặng; thứ tự biến và bộ tiền xử lý được kiểm tra khi nạp. Các kết quả notebook khác, kể cả HistGradientBoosting của BRFSS và greedy được chọn trong thí nghiệm RL, là kết quả so sánh nghiên cứu, chưa tự thay thế model MLP BRFSS và DQN đang dùng trên website. Toàn bộ biểu mẫu chỉ để chạy số liệu giả lập, không sàng lọc hay chẩn đoán.

4.3 Knowledge Base và RAG

Kho tri thức gồm đoạn tổng hợp nhập thủ công trong `knowledge/sources.json` và danh sách nguồn web trong `knowledge/web_sources.json`. Neo4j lưu node Document, Topic, WebSource cùng quan hệ ABOUT, FROM_SOURCE và vector embedding. Nomic Embed Text tạo embedding; truy vấn dùng cosine similarity, lọc ngưỡng và URL trùng. Mô hình `qwen3:4b-instruct` tạo câu trả lời tiếng Việt từ ngữ cảnh nhập thủ công.

Thu thập web là tác vụ riêng: kiểm tra HTTPS, host cho phép, robots.txt, kích thước và redirect; sau đó tạo embedding và cập nhật dữ liệu theo transaction. Nguồn web mang nhãn `unreviewed_web`; nguồn nhập thủ công cũng chưa được chuyên gia xác nhận. Cần kiểm chứng nội dung và kiểm thử truy xuất trước ứng dụng thực tế.

4.3.1 Các bước xây dựng và truy xuất tri thức

1. **Lựa chọn nguồn:** danh sách web là các URL cụ thể tại CDC/NIDDK; các tóm tắt thủ công có URL của nhiều tổ chức và bệnh viện. Mỗi mục có định danh để có thể cập nhật mà không tạo vô hạn bản sao.

2. **Tải và phân đoạn:** tác vụ thủ công kiểm tra host, robots và HTTP; trích heading, paragraph, list bên trong thẻ `main`. Các website không có cấu trúc phù hợp có thể không trích được nội dung. Tài liệu web được đánh dấu chưa kiểm duyệt.

3. **Embedding và lưu trữ:** Ollama tạo vector cho từng đoạn; Neo4j lưu vector cùng metadata, chủ đề và quan hệ. Khi nguồn lỗi, phiên bản đã lưu được giữ lại.

4. **Truy vấn:** hệ thống tạo embedding cho câu hỏi. Với câu hỏi tiếng Việt, hàm dịch sang tiếng Anh được dùng để tìm nguồn web; nếu dịch thất bại thì dùng câu gốc. Câu hỏi tiếp nối có thể được viết lại dựa trên các câu hỏi trước.

5. **Xếp hạng và sinh:** Neo4j tính cosine similarity, lấy ứng viên và lọc ngưỡng 0,875, loại trùng URL rồi giữ tối đa ba nguồn. Nội dung nhập thủ công được đưa vào prompt; nguồn web chưa duyệt không được đưa vào phần ngữ cảnh để sinh tóm tắt y tế.

Độ tương đồng cosine giữa vector q và d được tính theo công thức

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\|\|d\|}.$$

Điểm này chỉ đo mức gần trong không gian embedding; không chứng minh tài liệu đúng, cập nhật hoặc thực sự trả lời câu hỏi. Ngưỡng 0,875 là tham số triển khai, chưa được chứng minh tối ưu trên benchmark đại diện. Mô hình dịch và viết lại câu hỏi cũng có thể thay đổi nghĩa.

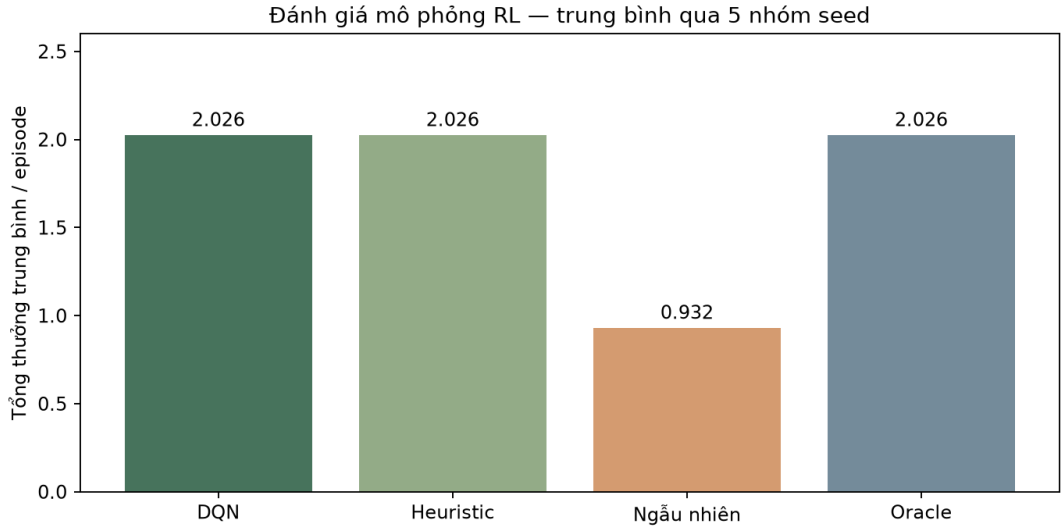
4.3.2 Kiểm soát ngữ cảnh và trích dẫn

Prompt hướng dẫn mô hình trả lời tiếng Việt, dựa vào nguồn, dùng ký hiệu số và không chỉ định liều. Lịch sử được đưa vào để hiểu ngữ cảnh, không được xem là tri thức. Các yêu cầu này là chỉ dẫn cho LLM, không phải bộ kiểm chứng hậu xử lý từng mệnh đề: mã hiện không đối chiếu độc lập từng câu với nguồn hoặc kiểm tra trích dẫn có thực sự hỗ trợ phát biểu. Vì vậy kết quả cần kiểm tra thủ công, nhất là với thông tin sức khỏe.

Nguồn tự động chỉ hiển thị liên kết nếu tất cả nguồn truy xuất đều thuộc nhóm chưa duyệt. Khi có cả nguồn thủ công và nguồn tự động, chỉ đoạn thủ công được đưa vào prompt sinh, còn nguồn tự động hiển thị riêng. Việc gắn nhãn "thủ công" không đồng nghĩa nội dung đã được chuyên gia thẩm định.

4.4 Reinforcement Learning: tác tử gợi ý học tập

Môi trường gồm bốn chủ đề. Trạng thái là bốn cờ nhị phân cần học, hành động chọn một chủ đề; phần thưởng $+1$ cho chủ đề cần học, $-0,25$ khi chọn lại, và 0 nếu tất cả đã hoàn tất. Một episode tối đa bốn bước. DQN được huấn luyện 30.000 timestep; đánh giá 200 episode cho mỗi nhóm trong 5 nhóm seed.



Hình 3: Tổng thưởng trung bình qua năm nhóm seed, lấy từ báo cáo RL đã lưu

DQN và heuristic đều đạt 2,026; ngẫu nhiên đạt 0,932. Trong bài toán đơn giản này DQN không cải thiện so với heuristic. Đây là môi trường giả lập và không chứng minh lợi ích y khoa.

4.4.1 Mô hình hóa bài toán DQN

Gọi $s_t \in \{0,1\}^4$ là trạng thái cần học và $a_t \in \{0,1,2,3\}$ là hành động chọn chủ đề. Khi $s_t[a_t] = 1$, hệ thống cho thưởng 1 và đặt thành phần đó về 0. Nếu chọn lại chủ đề đã hoàn tất trong khi còn nhu cầu thì phạt 0,25. Episode kết thúc khi cả bốn cờ bằng 0 hoặc đủ bốn bước. Trạng thái ban đầu được lấy ngẫu nhiên bằng bộ sinh số của môi trường.

DQN xấp xỉ hàm giá trị $Q(s,a)$ bằng mạng nơ-ron; về ý tưởng, mục tiêu TD sử dụng giá trị thưởng trước mắt cộng giá trị tương lai chiết khấu $r + \gamma \max_{a'} Q(s',a')$ khi chưa kết thúc. Mã sử dụng Stable-Baselines3, $\gamma = 0,99$, buffer 10.000, batch 64, bắt đầu học sau 200 bước và seed 42. Phần cài đặt replay buffer và cập nhật mạng do thư viện thực hiện.

Chính sách greedy trong bài toán đơn giản này luôn chọn chỉ số đầu tiên còn bằng 1, và oracle trong mã có cùng nguyên tắc chọn. Vì trạng thái giả lập cho biết trực tiếp chủ đề nào cần học, heuristic đã rất mạnh. Kết quả DQN bằng heuristic trên dữ liệu đánh giá đã lưu là một phép so sánh hữu ích nhưng không chứng minh DQN cần thiết cho ứng dụng thực.

4.5 Agent luật an toàn

Các mẫu luật nhận diện một số dấu hiệu khẩn cấp, yêu cầu điều chỉnh thuốc, thông tin định danh và prompt injection. Luật giúp định tuyến nhưng có thể bỏ sót hoặc nhầm; không thay thế nhân viên y tế. Khi không có nguồn phù hợp, hệ thống trả lời không tìm thấy; khi chỉ có nguồn tự động chưa duyệt, hệ thống trả liên kết.

4.5.1 Luật an toàn và các tình huống không chắc chắn

Rule Agent dùng biểu thức chính quy, không phân tích ngữ nghĩa sâu. Bộ luật có thể bỏ sót câu thiếu dấu, cách nói khác, thông tin được chia qua nhiều tin nhắn hoặc đưa ra cảnh báo nhầm với câu hỏi hợp lệ. Lịch sử không được kiểm tra lại toàn bộ bằng Rule Agent trong từng lượt; nội dung được sinh ra cũng chưa có bộ phân loại y khoa độc lập. Do đó không nên coi cơ chế này là lớp bảo vệ đầy đủ cho công khai hoặc xử lý thông tin nhạy cảm.

4.6 Điều phối và kết quả tự động

Trong nguyên mẫu, Coordinator điều phối luồng hỏi đáp và giao diện trình bày câu trả lời, nguồn cùng trace. Hệ thống không có bác sĩ trưởng, hội chẩn đa bác sĩ hay chức năng tự sinh báo cáo bệnh án. Báo cáo đồ án này được biên soạn từ mã, artifact và minh chứng thực tế, không phải đầu ra tự sinh của ứng dụng.

5 Code và Demo

5.1 Cấu trúc mã nguồn

- `backend/app/main.py`: API, định tuyến và các node LangGraph; `agents.py`: Rule Agent; `rag.py`: embedding, truy xuất và tạo câu trả lời.
- `ml/train.py`: làm sạch dữ liệu, huấn luyện MLP và baseline; `ml/rl.py`: môi trường DQN; `ml/seed_knowledge.py` và `ml/refresh_web.py`: nạp tri thức.
- `frontend/src/main.tsx`: giao diện React; `docker-compose.yml`: cấu hình dịch vụ; thư mục `artifacts/` chứa mô hình và báo cáo kết quả.
- Sáu notebook trong `notebooks/training` tách theo bài toán, có cell phân tích, làm sạch, huấn luyện, chọn model theo validation và đánh giá test; artifact mỗi lần chạy được lưu riêng.

5.2 Các hàm chính và quan hệ với giao diện

Hàm / vị trí	Đầu vào	Xử lý và kết quả
`validate_features`	Tên mô hình và dict đặc trưng	Kiểm tra khóa, phạm vi, mã hợp lệ; báo lỗi 422 nếu đầu vào không hợp lệ.

Hàm / vị trí	Đầu vào	Xử lý và kết quả
`RuleAgent.evaluate`	Câu hỏi	So khớp nhóm quy tắc theo thứ tự và tạo đối tượng kết quả có loại, lý do, thông báo.
`knowledge_node`	Câu hỏi và lịch sử	Viết lại câu hỏi nếu cần, truy xuất tối đa ba nguồn, cập nhật trace.
`rag.retrieve`	Câu hỏi, top-k, ngưỡng	Tạo embedding, truy vấn cosine trên Neo4j, lọc điểm và loại URL trùng.
`rag.answer_parts`	Câu hỏi, nguồn, lịch sử	Sinh các đoạn Ollama hoặc trả thông báo thiếu nguồn/nguồn tự động.
`train`	Nhánh, đường dẫn dữ liệu, thư mục đầu ra	Chia tập, fit tiền xử lý, huấn luyện baseline/MLP, lưu artifact và metrics.
`train_and_evaluate`	Số timestep và thư mục đầu ra	Huấn luyện DQN, đánh giá so với baseline và lưu JSON.

Trace được tạo tại các bước xử lý để người xem biết đường đi của một yêu cầu. Đối với phân loại và RL, tên các vai trò xuất hiện trong trace mô tả bước xử lý tại endpoint; không phải tất cả các bước đều là node trong cùng đồ thị LangGraph của chat.

5.3 Cách chạy và tái lập

1. Cài Docker Desktop và Ollama, kéo hai mô hình tên trong cấu hình `.env.example`. Tạo `.env` cục bộ với mật khẩu phù hợp; không chia sẻ nội dung tệp này.

2. Chạy `docker compose up --build -d` tại thư mục dự án. Chờ MySQL và Neo4j healthy trước khi API khởi động. Mở giao diện local tại cổng 8080 và kiểm tra endpoint health.

3. Chạy `docker compose exec api python -m ml.seed_knowledge` để nạp các bản tóm tắt thủ công. Khi muốn lấy nguồn web, chạy `docker compose --profile ingest run --rm knowledge-refresh`; xem log để xác nhận nguồn nào thành công.

4. Huấn luyện lại chỉ khi đã có dữ liệu và đủ tài nguyên; dùng notebook trong thư mục nộp hoặc lệnh trong README. Cần giữ môi trường thư viện tương thích và ghi lại phiên bản, seed, ngày chạy, hash dữ liệu và artifact.

5. Nếu dùng tunnel cấu hình sẵn, thêm tệp compose tunnel theo README. Kết nối từ bên ngoài còn phụ thuộc Cloudflare, chính sách truy cập, DNS, thiết bị host và mạng. Không đưa

credential vào báo cáo.

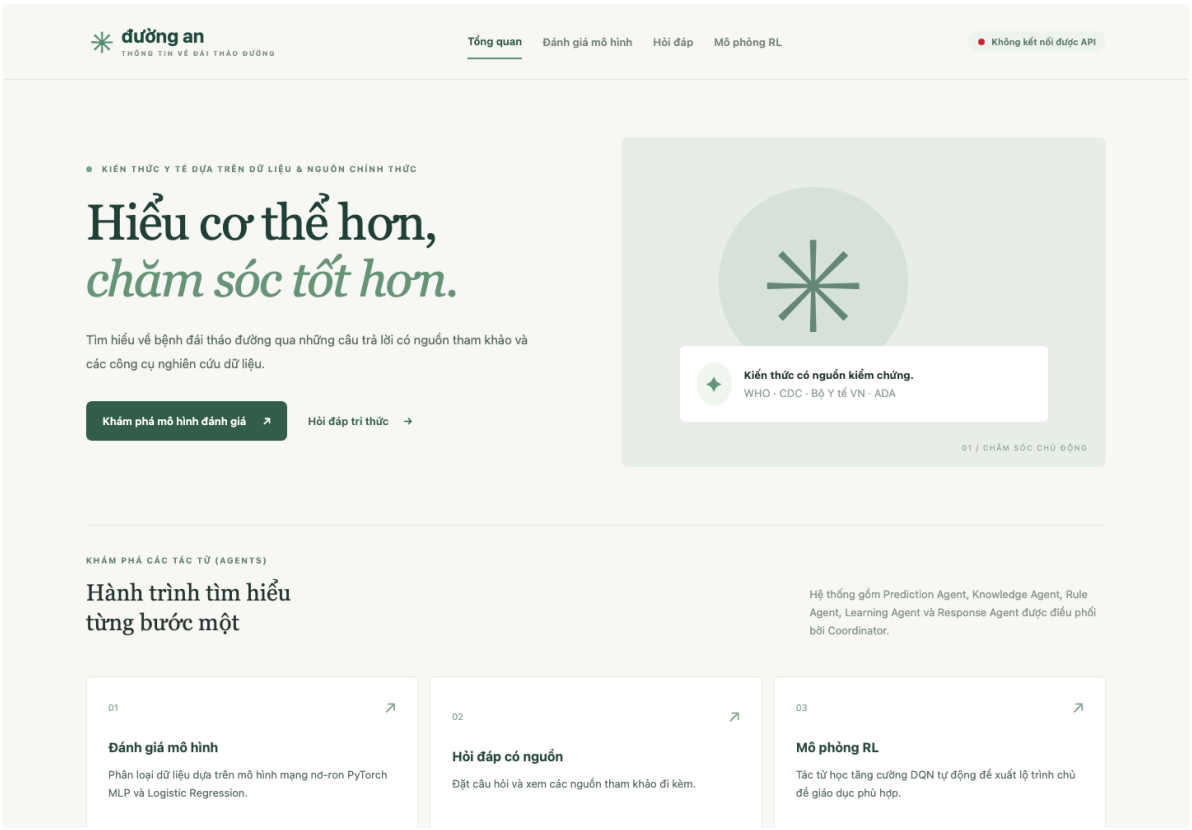
Kết quả ảnh chụp là trạng thái tại thời điểm chạy; khi dịch vụ tắt hoặc dữ liệu thay đổi, thao tác lặp lại có thể không cho cùng phản hồi LLM. Việc nạp tri thức trong Neo4j không đồng nghĩa nội dung đã được thẩm định y khoa.

5.4 Demo giao diện và triển khai

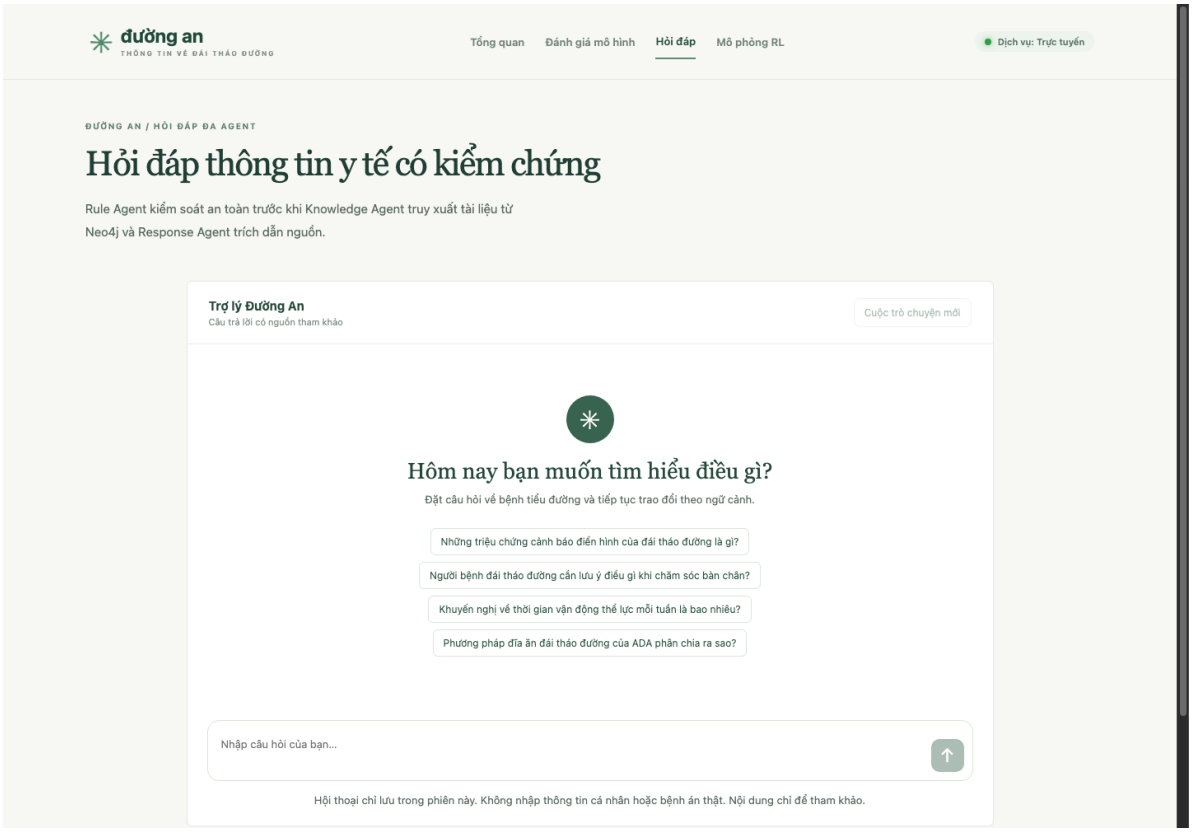
Địa chỉ local: `http://localhost:8080`; tài liệu API: `http://localhost:8001/docs`. Địa chỉ demo được cấu hình: `https://diabetes.handq2212.site`; mã nguồn GitHub: `https://github.com/HandQ2212/project-agent-diabets`. Ngày 05/11/2026, yêu cầu kiểm tra tự động nhận HTTP 403 từ Cloudflare; chưa xác nhận truy cập công khai bằng trình duyệt. Website phụ thuộc máy host, Docker Desktop và Ollama đang chạy, không phải dịch vụ luôn sẵn sàng.

Ngày 05/11/2026, danh sách có 18 đoạn thủ công và 25 nguồn web; sau lần nạp đã kiểm tra, Neo4j ghi nhận 25 nguồn và 190 đoạn web có embedding. Nguồn web vẫn chưa được kiểm duyệt chuyên môn. Các ảnh sau chụp ngày 04/11/2026, trước khi bổ sung biểu mẫu NHANES 15 chỉ số; chỉ minh họa giao diện phiên bản thời điểm chụp.

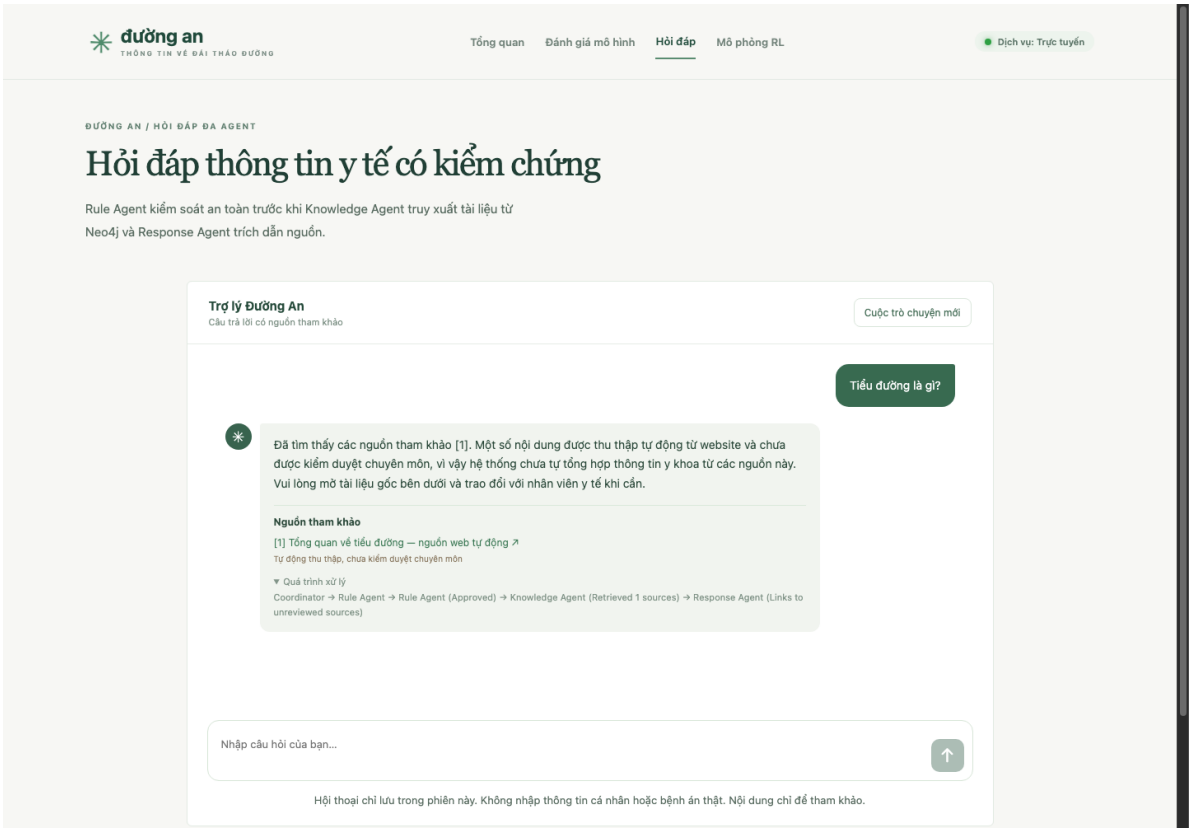
5.4.1 Minh chứng giao diện và trace



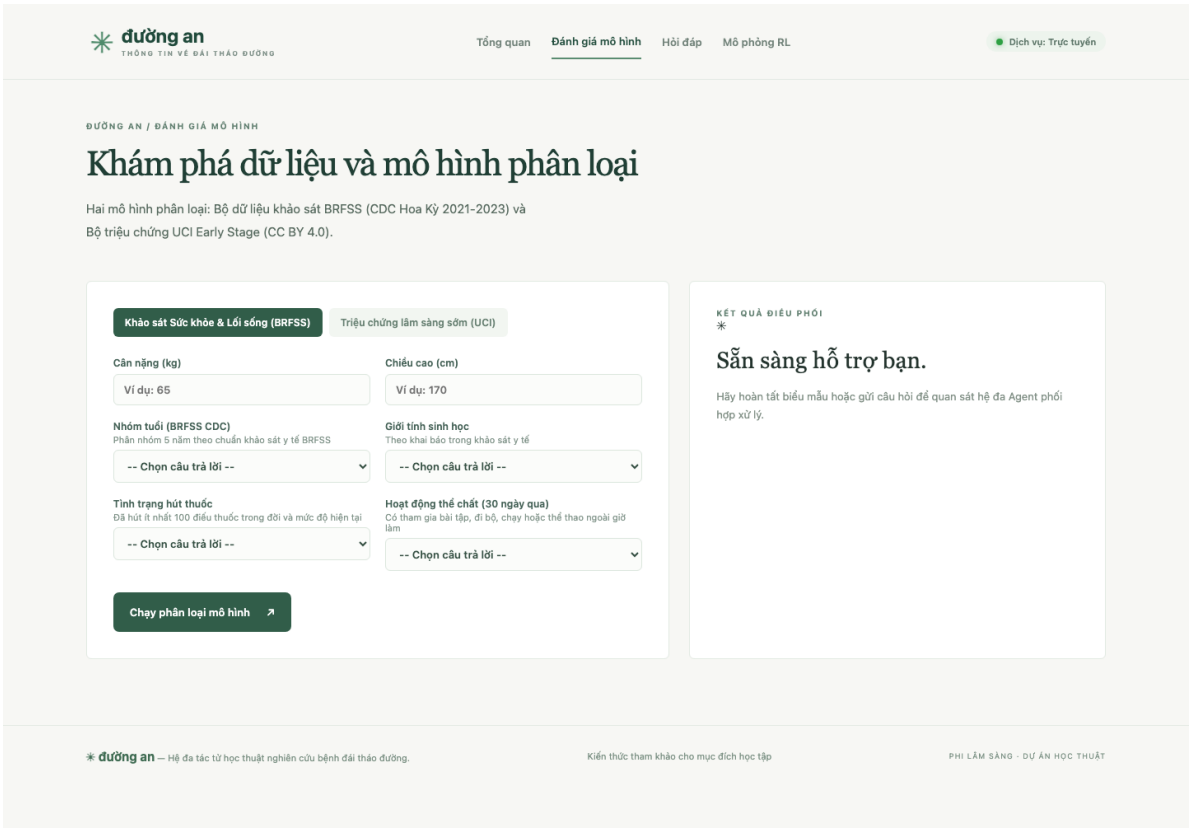
Hình 4: Trang chủ Đường An chụp từ ứng dụng đang chạy



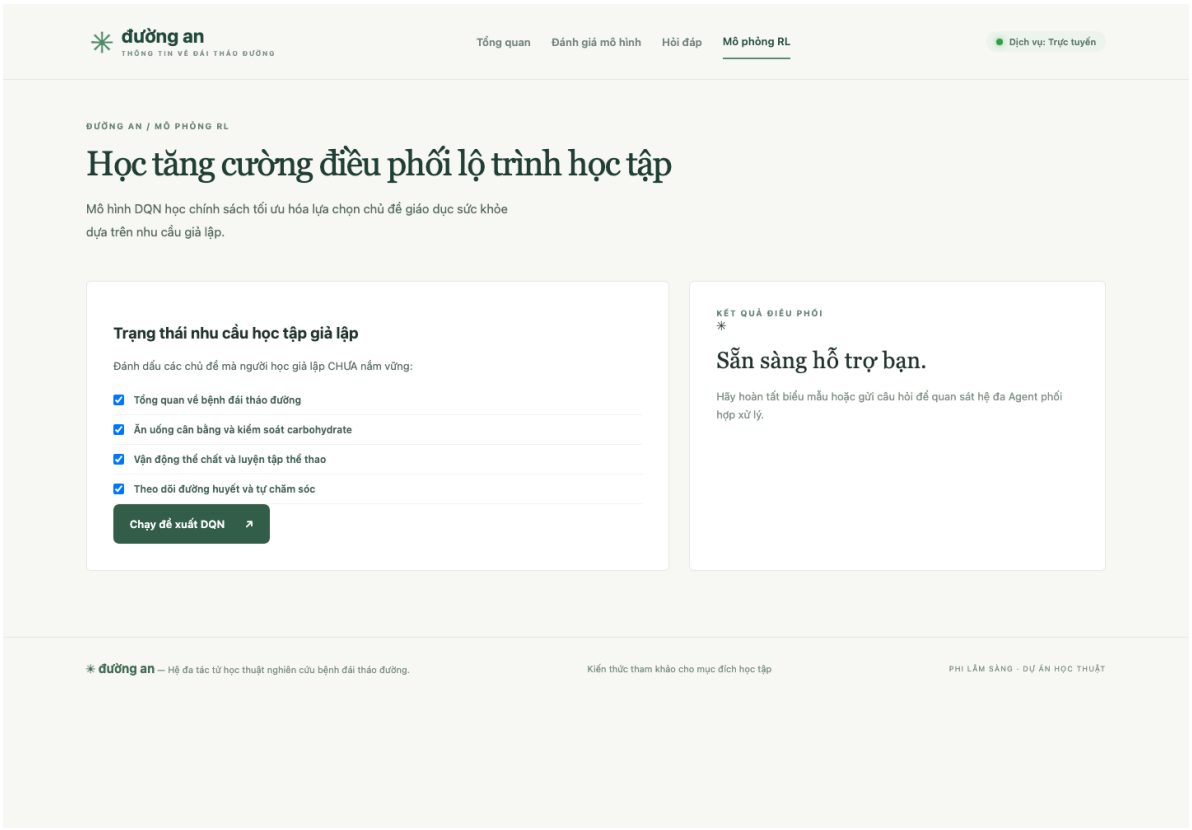
Hình 5: Giao diện nhập câu hỏi và các câu hỏi gợi ý



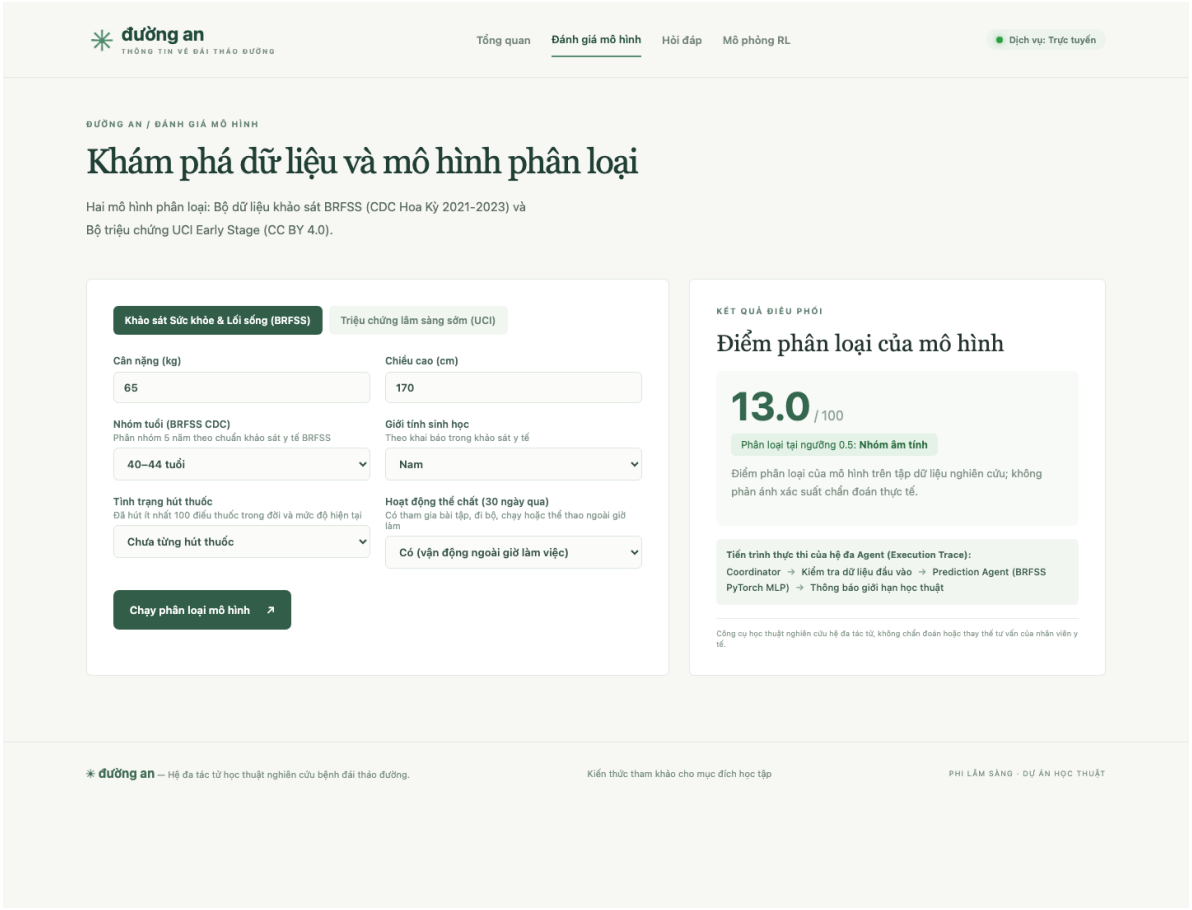
Hình 6: Ví dụ hỏi đáp thực tế: nguồn web chưa duyệt được hiển thị dạng liên kết; trace cho thấy các bước đã chạy



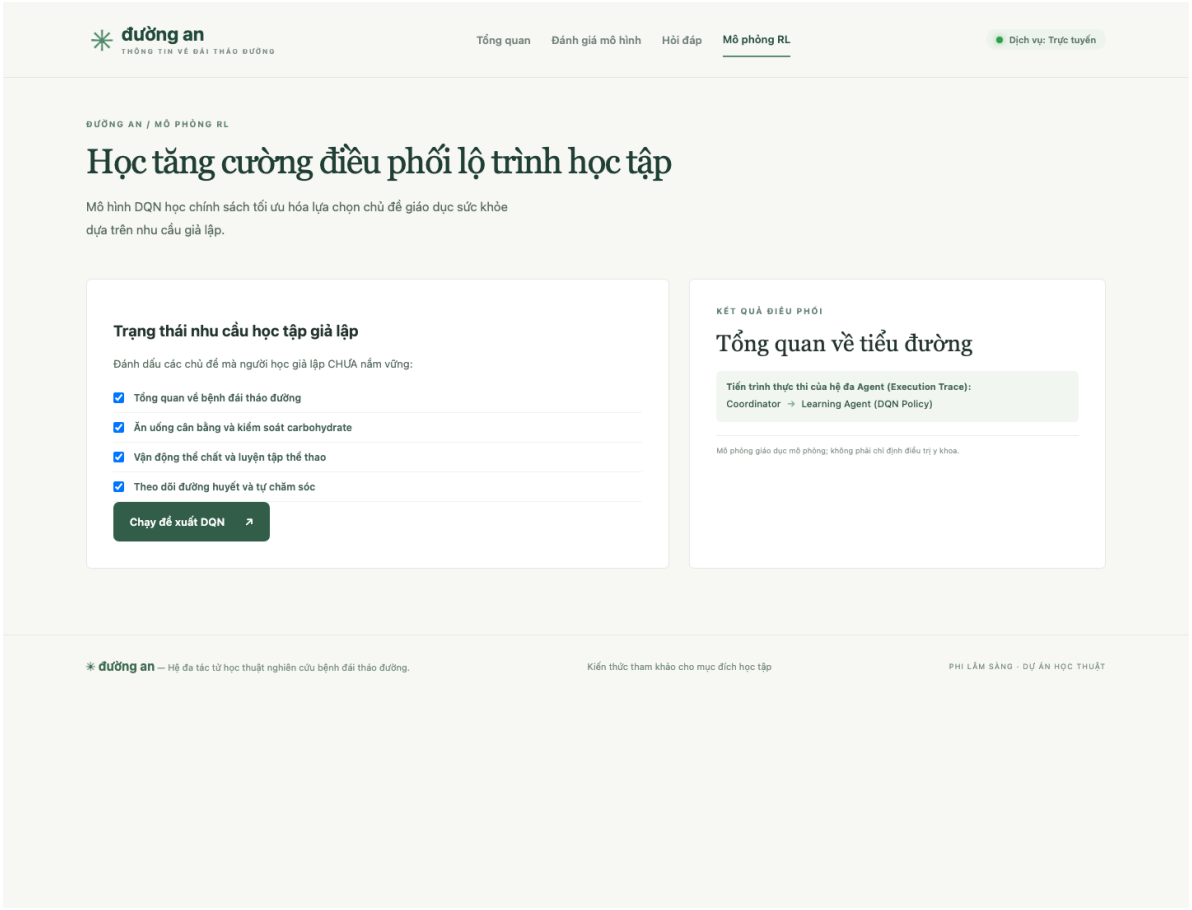
Hình 7: Giao diện phiên bản trước khi bổ sung biểu mẫu NHANES 15 chỉ số



Hình 8: Giao diện mô phỏng lựa chọn chủ đề giáo dục



Hình 9: Kết quả phân loại BRFSS với dữ liệu giả lập và trace Prediction Agent



Hình 10: Kết quả chọn chủ đề DQN với trạng thái giả lập và trace Learning Agent

Ảnh trace cho thấy Coordinator → Rule Agent → Knowledge Agent → Response Agent trên một yêu cầu cụ thể; đây là bằng chứng cho đường đi quan sát được, không chứng minh mọi trường hợp đầu vào đều an toàn hoặc chính xác.

5.4.2 Kết quả kiểm tra dịch vụ và lưu ý tái lập

Khởi động Ollama với hai mô hình `qwen3:4b-instruct`, `nomic-embed-text`; chạy `docker compose up --build -d`; nạp tài liệu thủ công và dùng profile ingest cho các nguồn web. Sáu notebook trong `notebooks/training` đã chạy đầy đủ và lưu các biểu đồ, đầu ra và báo cáo trong `artifacts/notebook_runs`. Các artifact này không tự cập nhật model website; mô hình NHANES 15 chỉ số trên web được sao chép cố định từ lần chạy đã chọn để bảo đảm đúng bộ tiền xử lý và ngưỡng. Không đưa mật khẩu, dữ liệu gốc hoặc credential tunnel vào hồ sơ.

5.5 Notebook thực hành

Sáu notebook tự chứa trong `notebooks/training` tách theo từng bài toán. Mỗi notebook thể hiện đọc và phân tích dữ liệu, làm sạch, chia tập, tiền xử lý, huấn luyện, lựa chọn theo validation,

đánh giá test và lưu artifact; các cell đã được thực thi và giữ đầu ra cùng biểu đồ. Notebook cũ `bao_cao_project/notebook/Training_Duong_An.ipynb` được giữ để tham khảo.

Các artifact notebook nằm ở `artifacts/notebook_runs`; chúng không tự ghi đè model ứng dụng. Việc chạy lại cần dữ liệu gốc, môi trường Python và thư viện tương thích; dữ liệu BRFSS lớn cần thời gian và bộ nhớ đáng kể. Snapshot model NHANES 15 chỉ số dùng cho web được lưu riêng trong `artifacts/nhanes15_web`.

5.6 Giải thích kết quả, phản biện và độ tin cậy

Giao diện hiển thị điểm mô hình, ngưỡng của model tương ứng và disclaimer; NHANES 15 dùng ngưỡng validation 0,8. Đây là giải thích loại đầu ra, không phải lời giải thích nguyên nhân dự đoán cho cá nhân. Hệ thống chưa cài SHAP, LIME, độ bất định theo cá nhân hay so sánh phản thực tế. Tương tự, trace cho biết các node đã được gọi, không đo độ đúng của nội dung.

Trong RAG, nguồn hiển thị giúp đối chiếu nhưng một trích dẫn hình thức không chứng minh mọi câu được hỗ trợ. Cần bổ sung bộ dữ liệu đánh giá gồm câu hỏi, tài liệu liên quan, đáp án tham chiếu và trường hợp không có câu trả lời; sau đó đo precision/recall của truy xuất, tính trung thực với nguồn, mức bao phủ trích dẫn và tỷ lệ từ chối đúng. Cần tách tập điều chỉnh ngưỡng và tập đánh giá cuối cùng.

Trong RL, so sánh với một baseline mạnh là điều cần thiết: kết quả DQN bằng heuristic cho thấy có thể giải bài toán giả lập mà không cần mô hình phức tạp hơn. Nếu mở rộng môi trường, phải xác định trạng thái quan sát, phản hồi người học và phần thưởng có ý nghĩa giáo dục, sau đó đánh giá thực nghiệm với sự đồng ý phù hợp; không thể suy ra hiệu quả chăm sóc sức khỏe từ reward hiện tại.

5.7 Kiểm thử và đánh giá kết quả

Nhóm	Điều được kiểm tra	Bằng chứng / giới hạn
Đầu vào	Thiếu trường, giá trị không hợp lệ, lịch sử quá dài	Kiểm thử API; không bao phủ mọi tổ hợp đầu vào.
Luật an toàn	Một số mẫu khẩn cấp, thuốc, PII, prompt injection	Có các ca định sẵn; không đo recall trên dữ liệu thực.
RAG	Có nguồn, có ký hiệu trích dẫn, câu hỏi ngoài lĩnh vực	Một ca ngoài lĩnh vực thất bại theo assertion trong lần chạy 04/11; có nguồn không liên quan được truy xuất.

Nhóm	Điều được kiểm tra	Bằng chứng / giới hạn
Stream	Chặn trước truy xuất; nhận các đoạn và nguồn cuối luồng qua mock	Kiểm tra hợp đồng; không đo độ đúng của văn bản sinh.
Dữ liệu	Schema UCI, chuyển nhãn, metric trường hợp một lớp	Dữ liệu giả lập trong test, chưa phải đánh giá thống kê toàn bộ pipeline.
Web ingest	HTML, redirect không tin cậy, kích thước và nguồn chưa duyệt	Mock HTTP; không thay thế kiểm tra bản quyền hoặc nội dung chuyên môn.

Trong lần chạy toàn bộ kiểm thử ngày 04/11, 23 test đạt và một test thất bại: câu hỏi nấu ăn truy xuất nguồn không liên quan, dù câu trả lời từ chối vì thiếu thông tin. Sau cập nhật biểu mẫu NHANES 15, ngày 05/11 chạy riêng các nhóm kiểm thử API, dữ liệu và nạp web: 29 test đạt, bao gồm kiểm tra thứ tự biến và ngưỡng của mô hình mới. Con số này không bao gồm bộ benchmark RAG tích hợp và không chứng minh chất lượng truy xuất hay chống ảo giác hoàn toàn.

5.8 Đánh giá và hạn chế

Về dữ liệu: UCI có tập test rất nhỏ; BRFSS sử dụng nhãn tự báo cáo và không dùng trọng số khảo sát. Khác biệt giữa đối tượng trong bộ dữ liệu và người dùng dự kiến có thể làm sai lệch kết quả. Không có đánh giá ngoài tại Việt Nam hay hiệu chỉnh xác suất cho người dùng ứng dụng.

Về tri thức: các bản tóm tắt thủ công và nguồn web cần thẩm định nội dung, thời điểm, quyền sử dụng và các đoạn được truy xuất. Nguồn web có thể thay đổi, xóa hoặc thay cấu trúc HTML. Không có bảo đảm trích dẫn là chính xác chỉ vì URL xuất hiện trên giao diện.

Về vận hành: health endpoint kiểm tra tồn tại tệp và kết nối, không xác minh toàn bộ khả năng suy luận. Rate limiter bộ nhớ không phù hợp thay thế hệ thống kiểm soát lưu lượng phân tán. Tunnel và các dịch vụ phụ thuộc thiết bị host; truy cập công khai có thể gặp thử thách Cloudflare. Các ảnh và kết quả là snapshot, không đại diện SLA hay đo tải.

5.9 Hướng phát triển

Ưu tiên ngắn hạn gồm xác minh bản tóm tắt với nguồn gốc, sửa và mở rộng benchmark ngoài lĩnh vực, bổ sung tập đánh giá RAG có gán nhãn, cải thiện kiểm tra lời sinh và xây dựng quy trình rà soát nội dung. Với mô hình dữ liệu, cần lựa chọn mục tiêu phù hợp, xác minh nguồn dữ liệu cũ, đánh giá ngoài, khảo sát hiệu chỉnh và báo cáo bất định. Với sản phẩm, cần kiểm thử giao diện, quyền riêng tư, cấu hình reverse proxy, giới hạn truy cập và giám sát lỗi trước bất kỳ triển khai

rộng nào.

6 Kết luận

Đồ án đã tích hợp các nhánh hỏi đáp RAG, mô hình phân loại và mô phỏng DQN trong một giao diện. Các minh chứng gồm ảnh chụp giao diện, trace node, báo cáo định lượng và notebook có mã nguồn. Giới hạn chính gồm chất lượng nguồn chưa được chuyên gia thẩm định, quy tắc an toàn chỉ theo mẫu, mô hình không được hiệu chỉnh/kiểm chứng ở Việt Nam, cùng sự phụ thuộc máy chủ cá nhân. Cần đánh giá RAG bằng bộ câu hỏi có đáp án, thẩm định y khoa, kiểm thử bảo mật, kiểm định ngoài và xác định rõ mục tiêu dữ liệu trước khi mở rộng.

Tài liệu tham khảo

1. CDC BRFSS, dữ liệu và codebook 2021–2023: https://www.cdc.gov/brfss/annual_data/annual_data.htm.
2. UCI Machine Learning Repository, Early Stage Diabetes Risk Prediction: https://archive.ics.uci.edu/dataset/529/early+stage+diabetes+risk+prediction.
3. CDC, Diabetes Meal Planning: https://www.cdc.gov/diabetes/healthy-eating/diabetes-meal-planning.html.
4. NIDDK, Preventing Type 2 Diabetes: https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-type-2-diabetes.
5. Vinmec, Tổng quan về bệnh đái tháo đường: https://www.vinmec.com/vie/bai-viet/tong-quan-ve-benh-dai-thao-duong-vi.
6. FPT Long Châu, Những câu hỏi thường gặp về bệnh tiểu đường: https://nhathuoclongchau.com.vn/bai-viet/tim-hieu-loi-giai-cho-nhung-cau-hoi-thuong-gap-o-benh-tieu-duong.html.
7. CDC, NHANES 2017–2018: https://wwwn.cdc.gov/nchs/nhanes/continuousnhanes/default.aspx?BeginYear=2017.
8. WHO, Viet Nam STEPS 2021, metadata: https://extranet.who.int/ncdsmicrodata/index.php/catalog/948. Chưa tải microdata này và không dùng để huấn luyện website.
9. Mã dự án `ml/`, `backend/`, `knowledge/`; các tệp `artifacts/*/report.json`, truy cập ngày 04/11/2026.